FINFLOW – Personal Finance Management System

Submitted for
UCS310 – Database Management System

Submitted By
Suryansh Malik (1024170308)
Tanmay Agarwal (1024170309)
B.Tech (2nd Year)

Lab Instructor
Ms. Tanya Garg

1. Introduction

In today's world, managing personal finances has become increasingly complex due to multiple income sources, diverse spending categories, various payment methods, and different financial goals. Traditional methods like maintaining physical records or using basic spreadsheets are proving inadequate as they create scattered data, make it difficult to analyze spending patterns, and lack automation for routine financial tasks.

FinFlow addresses these challenges by offering a database-driven solution for comprehensive personal finance management. The system maintains a centralized repository of financial information including bank accounts, credit cards, daily transactions, spending categories, monthly budgets, savings goals, and recurring payments.

This project was chosen to explore how financial applications leverage database systems to maintain accuracy, ensure security, and provide meaningful insights. A DBMS-based approach offers significant advantages over file-based systems: it prevents data duplication, ensures consistency across related information, enables complex financial analysis through queries, supports multiple users (like family members) accessing shared data, and provides robust transaction handling for critical operations like fund transfers.

2. Problem Statement

Currently, individuals managing their finances face several critical challenges:

- Financial data scattered across notebooks, receipts, and multiple spreadsheet files
- Inability to get consolidated view of all accounts and transactions
- Manual effort required for categorizing expenses and tracking budgets
- No automatic alerts when spending exceeds planned budget limits
- Difficulty in calculating key financial metrics like savings rate or debt-to-income ratio
- Lack of automated processing for recurring bills and salary deposits
- Time-consuming manual reconciliation between bank statements and personal records
- Risk of calculation errors when dealing with splits, transfers, and currency conversions

While commercial applications like Mint and YNAB exist, they function as black boxes without revealing their internal database architecture. For academic learning, we need a system that exposes the complete database design process - from entity identification to normalization, from constraint definition to trigger implementation.

FinFlow bridges this gap by building a finance management system from the ground up, focusing on robust database design, proper normalization, effective use of SQL for data manipulation, and intelligent automation through PL/SQL components.

3. Objectives of the Project

- Design a comprehensive database schema covering all aspects of personal finance - users, accounts, transactions, categories, budgets, goals, debts, and recurring payments
- Create an Entity-Relationship diagram identifying entities, their attributes, relationships between them, and appropriate cardinality constraints
- Transform the ER model into a normalized relational structure with properly defined primary keys, foreign keys, and integrity constraints
- Apply normalization principles (1NF through 3NF/BCNF) to eliminate data redundancy and resolve functional dependencies
- Implement complex SQL queries demonstrating joins across multiple tables, nested subqueries, aggregation functions, grouping and filtering, and reusable views
- Develop PL/SQL automation including procedures for multi-step operations, functions for financial calculations, triggers for automatic updates and validations, and cursors for report generation
- Demonstrate transaction management principles ensuring ACID properties during critical operations like account transfers and payment processing
- Build an automated financial tracking system where recurring transactions process automatically, account balances update in real-time, budget overruns trigger alerts, and goal progress tracks continuously

4. Scope of the Project

User Roles:

Individual User: Manages personal accounts, records income and expenses, sets category budgets, defines savings goals, and analyzes spending patterns.

Family Administrator: Oversees shared household accounts, allocates budgets across family members, tracks joint expenses, and generates family-wide financial reports.

System Admin: Maintains database integrity, manages user permissions, monitors system performance, and generates administrative reports.

System Modules:

- User & Authentication Module - Registration, login, profile management, and access control

- Account Module - Multiple account types (savings, checking, credit card, cash, e-wallet) with balance tracking
- Transaction Module - Recording income, expenses, and transfers with categorization and notes
- Category Module - Hierarchical organization of income and expense categories
- Budget Module - Setting spending limits per category with tracking against actuals
- Goal Module - Defining financial targets with deadline and progress monitoring
- Recurring Payment Module - Automating regular transactions like salary, rent, and subscriptions
- Debt Module - Tracking loans with EMI calculations and payoff schedules
- Analytics & Reporting Module - Generating insights through queries and stored procedures

Note: This project emphasizes backend database implementation. The focus is on schema design, data integrity, query optimization, and PL/SQL automation rather than user interface development.

5. Proposed System Description

FinFlow operates as a relational database system with the following workflow:

- Users register and create multiple financial accounts representing their real-world banking relationships
- Every financial activity (income received, expense paid, money transferred) is logged as a transaction
- Transactions are automatically categorized (or manually assigned) to spending/income categories
- Monthly or yearly budgets can be set for any category to control spending
- Database triggers automatically update account balances whenever transactions are added or modified
- When spending in a category exceeds its budget, alerts are generated through triggers
- Recurring transactions (like monthly salary or rent) are processed automatically by scheduled procedures
- Financial goals are tracked, with progress calculated based on relevant transactions
- Inter-account transfers are handled atomically - both debit and credit operations must succeed together
- Stored procedures handle complex multi-step operations ensuring consistency
- Functions compute derived metrics like net worth, savings percentage, and debt ratios
- Views provide pre-defined perspectives on data for common reporting needs

Key Benefits:

- Eliminates manual calculations through automated triggers and functions
- Maintains perfect consistency between accounts and transactions via referential integrity
- Provides instant insights through optimized queries and indexed columns
- Prevents invalid states (like negative balances) through check constraints
- Creates complete audit trail of all financial activities
- Supports consolidated reporting across multiple accounts

6. Database Design

The database comprises nine core entities:

User: Maintains user profiles with UserID (primary key), full name, email address (unique), phone number, date of birth, and account creation timestamp. Forms the central entity linking to all financial data.

Account: Represents individual financial accounts with AccountID (PK), reference to owner via UserID (FK), descriptive account name, account type indicator (Savings/Checking/CreditCard/Cash/Wallet), currency code, current balance, and active status flag.

Transaction: Records every financial activity through TransactionID (PK), associated account via AccountID (FK), expense/income category via CategoryID (FK), transaction amount, date-time stamp, transaction type (Income/Expense/Transfer), optional description, and payment method used.

Category: Organizes transactions hierarchically with CategoryID (PK), category name, optional parent category via ParentCategoryID (FK to self), and category type (Income vs Expense). Enables nested categorization like Food → Groceries → Vegetables.

Budget: Tracks spending limits through BudgetID (PK), user assignment via UserID (FK), target category via CategoryID (FK), allocated amount, time period (Monthly/Yearly), validity start date, and end date.

Goal: Manages financial objectives with GoalID (PK), owner via UserID (FK), goal description, target amount to achieve, current saved amount, target completion date, and status indicator (Active/Achieved/Cancelled).

RecurringTransaction: Handles repetitive payments via RecurringID (PK), source account via AccountID (FK), category via CategoryID (FK), transaction amount, recurrence frequency (Daily/Weekly/Monthly/Yearly), next scheduled date, and description text.

Debt: Monitors loans and liabilities through DebtID (PK), borrower via UserID (FK), debt description, original principal amount, annual interest rate, monthly EMI payment, loan start date, maturity date, and outstanding balance.

TransactionSplit: Enables single transaction distribution across multiple categories using SplitID (PK), parent transaction via TransactionID (FK), allocated category via CategoryID (FK), and portion amount. Example: grocery bill split between Food and Household categories.

These entities are interconnected through foreign key relationships ensuring referential integrity - any change to master data automatically reflects in dependent records.

7. Relational Schema

Complete table structures with keys and relationships:

Underlined attributes are primary keys, asterisked (*) attributes are foreign keys.

- User(UserID, Name, Email, Phone, DateOfBirth, CreatedDate)
- Account(AccountID, UserID*, AccountName, AccountType, Currency, CurrentBalance, IsActive)
- Category(CategoryID, CategoryName, ParentCategoryID*, CategoryType)
- Transaction(TransactionID, AccountID*, CategoryID*, Amount, TransactionDate, TransactionType, Description, PaymentMethod)
- Budget(BudgetID, UserID*, CategoryID*, BudgetAmount, Period, StartDate, EndDate)
- Goal(GoalID, UserID*, GoalName, TargetAmount, CurrentAmount, TargetDate, Status)
- RecurringTransaction(RecurringID, AccountID*, CategoryID*, Amount, Frequency, NextDueDate, Description)
- Debt(DebtID, UserID*, DebtName, PrincipalAmount, InterestRate, MonthlyEMI, StartDate, EndDate, RemainingBalance)
- TransactionSplit(SplitID, TransactionID*, CategoryID*, SplitAmount)

Constraint Implementation:

- Primary Keys: Auto-incrementing unique identifiers for all entities
- Foreign Keys: Enforce relationships with ON DELETE CASCADE for dependent data, ON DELETE RESTRICT for referenced data
- Unique Constraints: Email addresses must be unique across all users
- Check Constraints: Transaction amounts must be positive, dates must be valid, account types must match predefined values
- Not Null Constraints: Critical fields like Amount, TransactionDate, AccountType cannot be empty
- Default Values: Transaction date defaults to current timestamp, IsActive defaults to true

8. Normalization

Functional Dependencies Identified:

Key functional dependencies in the system:

- UserID → Name, Email, Phone, DateOfBirth (User attributes depend on UserID)
- AccountID → UserID, AccountName, AccountType, Balance (Account properties determined by AccountID)
- TransactionID → AccountID, CategoryID, Amount, Date, Type (Transaction details depend on TransactionID)
- CategoryID → CategoryName, ParentCategory, Type (Category characteristics determined by CategoryID)
- BudgetID → UserID, CategoryID, Amount, Period (Budget specifications depend on BudgetID)
- Email → UserID (Email uniquely identifies a user)

Normalization Steps:

First Normal Form (1NF):

Achieved by ensuring atomic values in all columns. Initially, transactions might have stored multiple categories as comma-separated text - this was resolved by creating the TransactionSplit table where each category gets its own row. Similarly, storing multiple phone numbers in a single field would violate 1NF; instead, we use a single phone field or create a separate ContactInfo table if multiple numbers are needed.

Second Normal Form (2NF):

Achieved by eliminating partial dependencies. All non-key attributes must depend on the complete primary key. In the Transaction table, attributes like Amount and Date depend fully on TransactionID, not on just part of it. Category information (CategoryName, Type) is separated into its own Category table rather than being embedded in Transaction, as it depends on CategoryID alone.

Third Normal Form (3NF):

Achieved by removing transitive dependencies. Non-key attributes should not depend on other non-key attributes. For example, if we stored both City and State in User table, and State could be derived from City, that would be a transitive dependency. We resolve this by separating location data into a dedicated table. Account balance is technically derivable from summing transactions, but we store it for performance (denormalization) and maintain consistency through triggers.

Boyce-Codd Normal Form (BCNF):

Ensures every determinant is a candidate key. In our schema, all determining attributes are either primary or unique keys. The RecurringTransaction table maintains BCNF as

Frequency determines calculation logic for NextDueDate, but this is handled through procedural code rather than storing derived data.

Final Result:

The normalized design eliminates redundancy, reduces update anomalies, maintains consistency through well-defined relationships, and ensures that each piece of information is stored exactly once in its appropriate location.

9. Database Implementation

9.1 SQL Implementation

DDL (Data Definition Language):

- CREATE TABLE statements defining structure for all nine entities with appropriate data types
- ALTER TABLE commands for schema modifications like adding new columns or modifying constraints
- DROP TABLE statements for removing obsolete structures
- CREATE INDEX on frequently searched columns (UserID, TransactionDate, CategoryID) for performance
- CREATE VIEW definitions for commonly used data perspectives

DML (Data Manipulation Language):

- INSERT operations for adding users, accounts, transactions, budgets, and goals
- UPDATE statements for modifying balances, budget amounts, goal progress, and user profiles
- DELETE operations for removing completed goals, old transactions, or closed accounts
- SELECT queries for data retrieval and analysis

Advanced SELECT Operations:

Complex Joins: **Combining data from User, Account, Transaction, Category, and Budget tables to generate comprehensive financial reports. Example: joining Transaction with Category and Budget to show spending vs. limits by category.**

Subqueries: Nested queries to find users spending above average, accounts with highest transaction counts, or categories exceeding budget most frequently.

Aggregate Functions: SUM for total income/expenses, AVG for average transaction amounts, COUNT for transaction frequency, MIN/MAX for spending ranges.

GROUP BY & HAVING: Grouping transactions by category, month, or account type, then filtering groups based on aggregate conditions like total spending > budget limit.

Views: Pre-defined queries for account summary, monthly expense breakdown, budget status, goal progress, and transaction history.

9.2 PL/SQL Implementation

Stored Procedures:

- sp_TransferFunds(fromAcct, toAcct, amount) - Atomically moves money between accounts with validation
- sp_ProcessRecurringTransactions() - Batch processes all due recurring transactions
- sp_SplitTransaction(transID, categories[], amounts[]) - Distributes transaction across multiple categories
- sp_UpdateGoalProgress(goalID) - Recalculates goal completion percentage
- sp_GenerateMonthlyReport(userID, month) - Compiles spending summary by category

Functions:

- fn_CalculateNetWorth(userID) - Returns assets minus liabilities
- fn_GetSavingsRate(userID, month) - Computes (Income - Expenses) / Income as percentage
- fn_CheckSufficientBalance(accountID, amount) - Returns boolean if balance adequate
- fn_CalculateEMI(principal, rate, months) - Computes monthly installment amount
- fn_GetCategoryTotal(categoryID, startDate, endDate) - Sums transactions in category for period

Triggers:

- trg_UpdateBalance_AfterInsert - Updates account balance when new transaction added
- trg_UpdateBalance_AfterUpdate - Adjusts balance when transaction amount modified
- trg_UpdateBalance_AfterDelete - Reverses balance change when transaction removed
- trg_CheckBudget_AfterTransaction - Generates alert if category spending exceeds budget
- trg_PreventNegativeBalance - Blocks updates that would create negative balance in non-credit accounts
- trg_ValidateTransactionDate - Ensures transaction dates are not in future
- trg_AutoCategorize - Assigns category based on merchant name patterns
- trg_LogAudit - Records all data modifications for audit trail

Cursors:

Used extensively in reporting procedures to iterate through result sets: processing all recurring transactions due today, generating category-wise expense reports, calculating running balances across date ranges, and aggregating data for analytics.

Exception Handling:

Every procedure includes error handling blocks to catch constraint violations, insufficient balance errors, invalid date ranges, and division by zero. Failed operations trigger ROLLBACK to maintain database consistency.

10. Transaction Management & Concurrency

Transaction management is fundamental to financial data integrity. The system implements ACID properties:

Atomicity: Multi-step operations like fund transfers execute completely or not at all. If debiting the source account succeeds but crediting the destination fails, the entire operation rolls back.

Consistency: Database moves from one valid state to another. Account balances always match transaction totals. Budget allocations always reconcile with actual spending records.

Isolation: Concurrent users don't interfere with each other. If two family members access the same account simultaneously, their transactions process independently through proper locking.

Durability: Once committed, transactions persist even if the system crashes. Financial records are permanently stored and recoverable.

Transaction Control Implementation:

BEGIN TRANSACTION: Marks the start of atomic operation block

COMMIT: Permanently saves all changes when operation succeeds

ROLLBACK: Undoes all changes if any step fails

SAVEPOINT: Creates intermediate checkpoint for partial rollback in complex operations

Critical Scenarios:

- Account transfers require atomic debit-credit pair
- Split transactions need all category allocations to succeed together
- Budget updates must recalculate spending atomically
- Recurring transaction processing must handle all due items as a unit
- Debt payment recording must update both debt balance and account balance together

11. Tools & Technologies Used

Database Management System: Oracle Database 11g/19c, MySQL 8.0, or PostgreSQL 13 for implementing relational schema, enforcing constraints, and storing financial data securely.

SQL (Structured Query Language): Used for schema definition (DDL), data manipulation (DML), querying and analysis, constraint enforcement, and view creation.

PL/SQL / Stored Procedures: Oracle PL/SQL or MySQL stored procedure language for implementing business logic, automation, calculations, validation, and error handling.

Development Environment: Oracle SQL Developer, MySQL Workbench, or pgAdmin for visual schema design, query development, debugging, and performance tuning.

Version Control: Git for tracking schema changes and SQL script versions.

12. Expected Outcomes

- A fully normalized database schema free from redundancy and update anomalies
- Efficient query performance through strategic indexing and optimized SQL
- Automated financial operations reducing manual effort and errors
- Robust data integrity maintained through constraints, triggers, and transactions
- Comprehensive financial analytics providing actionable insights on spending, savings, and goal progress
- Deep practical understanding of database design principles, normalization theory, SQL programming, and PL/SQL automation
- Real-world applicable system demonstrating how commercial financial applications are structured at the database level
- Experience with transaction management, concurrency control, and ACID properties in critical financial operations

This project demonstrates end-to-end database application development - from conceptual design through physical implementation to automated operation - using a real-world financial domain that requires strict data integrity and sophisticated query capabilities